Int. Artificial y Redes Neuronales

Resolución de Problemas mediante Búsqueda

Dr. Mario Angel Rico Mendez

28 de octubre de 2026

FIME
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA



Table of Contents

1 Agentes que resuelven problemas

- Agentes que resuelven problemas
- Algoritmos de búsqueda

Agentes que resuelven problemas —Motivación

- Imagina un agente de viaje en Rumania.
- El agente desea: conocer lugares, mejorar su rumano, disfrutar la vida nocturna.
- El problema de decisión es complejo.
- Supón que el agente está en **Arad** y tiene un vuelo **no reembolsable** desde **Bucarest** al día siguiente.
- Si el entorno es **desconocido**, el agente sólo puede ejecutar acciones al azar.

Mapa del problema

- **Hoja de ruta**: desde la formulación del *goal* hasta el plan/solución.
- Este tema usa el **mapa de Rumania** como ejemplo canónico de rutas.

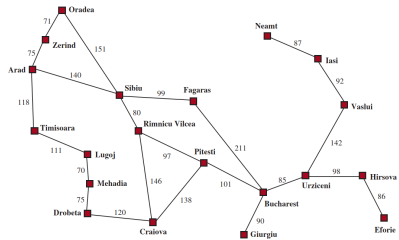


Figura 1: A simplified road map of part of Romania

Formulación de metas (*Goal formulation*)

- El agente adopta la meta de **llegar a Bucarest**.
- Las metas organizan la conducta al **restringir objetivos y acciones** a considerar.

Formulación de problemas (*Problem formulation*)

- El agente diseña una **descripción de estados y acciones** para alcanzar la meta.
- Es un **modelo abstracto** de la parte relevante del mundo.

Búsqueda

- Antes de actuar en el mundo real, el agente **simula secuencias** de acciones en su modelo.
- Busca hasta hallar una **secuencia de acciones** que alcance la meta.
- Dicha secuencia es una **solución**.

Definición formal de un problema de búsqueda

Un problema de búsqueda consiste en:

- **Espacio de estados**: conjunto de estados posibles del entorno.
- **Estado inicial**: donde comienza el agente (p.ej., Arad).
- **Estados meta**: uno o varios (p.ej., Bucarest).
- **Acciones**: conjunto disponible; ACTIONS(s) da las aplicables en s.
- **Modelo de transición**: RESULT(s, a) devuelve el estado resultante.
- **Costo de acción**: $c(s, a, s')$ o ACTION_COST(s, a, s').

Caminos y soluciones

- Una **secuencia de acciones** forma un **camino**.
- Una **solución** es un camino desde el estado inicial hasta un estado meta.
- Los costos de acción son **aditivos**: el costo del camino es la suma de costos.
- Una **solución óptima** tiene el **menor costo de camino**.

Representación del espacio de estados

- Se puede representar como **grafo**:
  - Vértices: **estados**
  - Arcos dirigidos: **acciones**
- Ejemplo: en *route-finding*, **ciudades = estados** y **carreteras = acciones**.

Ejemplo: búsqueda de ruta en Rumania

- **Inicial**: Arad **Meta**: Bucarest
- **Acciones**: viajar de una ciudad a otra.
- **Transición**: RESULT(s, a) retorna la ciudad destino.
- **Costo**: distancia en millas entre ciudades.
- **Solución**: un camino Arad→Bucarest con **menor distancia total**.

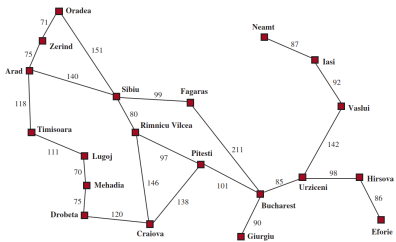


Figura 2: Ruta en Rumania

Section 1.1

Formular problemas

Formular problemas: abstracción

- La formulación es una **descripción matemática abstracta**, no el mundo real.
- Ej.: "*Estar en Arad*" simplifica un viaje real a través del país.
- Detalles **irrelevantes se omiten** para enfocarse en el objetivo (p.ej., ruta a Bucarest).
- Este proceso se llama **abstracción**. Una **buena** formulación tiene el **nivel adecuado** de detalle.

Abstracción (validez)

- Estados y acciones **abstractos** corresponden a grandes conjuntos de estados y secuencias detallados.
- Es válida si: *cada* estado detallado en un estado abstracto (p.ej., “en Arad”) tiene un **camino detallado** a otro estado abstracto (p.ej., “en Sibiu”).
- Garantiza que las **soluciones abstractas** puedan **elaborarse** en soluciones detalladas.

Problemas de ejemplo

- **Estandarizados**: ilustran métodos (p.ej., *grid world*).
- **Del mundo real**: idiosincrásicos y específicos (p.ej., navegación robótica).

Problemas estandarizados: *Grid World*

- Arreglo rectangular 2D de celdas cuadradas.
- El agente se mueve a celdas adyacentes (H/V o diagonal).
- Ejemplo: **Vacuum World**.

Vacuum World como problema de grilla

Estados:

- Qué objetos están en qué celdas (agente y suciedad).
- En mundo de 2 celdas: $2 \cdot 2 \cdot 2 = 8$ estados.
- En mundo de n celdas: $n \cdot 2^n$ estados.

Inicial: cualquiera. **Acciones**:

- 2 celdas: Aspirar, Izq, Der.
- Multi-celda: agregar Arriba, Abajo o egocéntricas (Adelante, Atrás, GiroDer, GiroIzq).

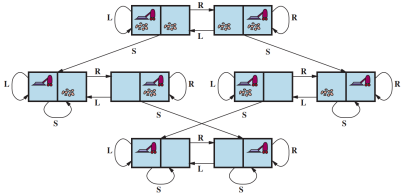


Figura 3: The state-space graph for the two-cell vacuum world

Vacuum World: modelo de transición

- Aspirar: remueve suciedad de la celda del agente.
- Adelante/Atrás: mueve una celda (si no hay pared).
- GiroDer/GiroIzq: rota 90° la orientación.

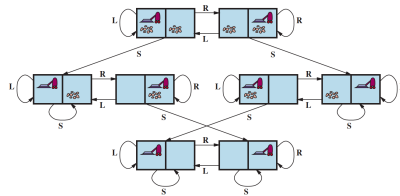


Figura 4: The state-space graph for the two-cell vacuum world

Vacuum World: metas y costo

- **Metas**: todas las celdas limpias.
- **Costo de acción**: cada acción cuesta 1.

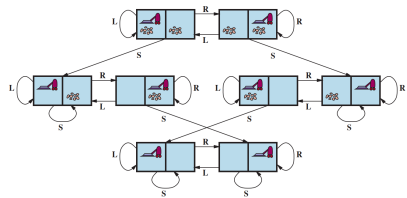


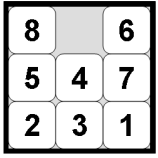
Figura 5: The state-space graph for the two-cell vacuum world

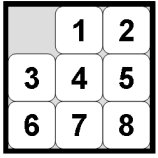
Section 1.2

Rompecabezas deslizante (8-Puzzle)

8-Puzzle: definición

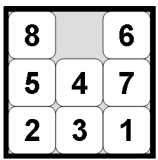
- Fichas en una grilla con uno o más **espacios en blanco**.
- Las fichas **se deslizan** hacia el espacio en blanco.
- La meta es alcanzar un **estado objetivo** especificado.

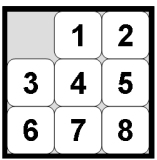




8-Puzzle: formulación estándar

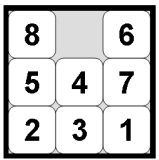
- **Estados**: especifican la ubicación de cada ficha.
- **Inicial**: cualquier estado; una propiedad de **paridad** parte el espacio de estados: solo **la mitad** de los estados iniciales pueden alcanzar una meta dada.
- **Acciones**: el **blanco** se mueve Izq/Der/Arr/Abj; en bordes/esquinas, algunas no aplican.

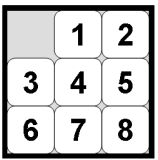




8-Puzzle: transición, meta y costo

- **Transición**: mapea (estado, acción) $\rightarrow$ estado. Ej.: aplicar IZQ al inicio intercambia el 5 y el blanco.
- **Meta**: típicamente, números en orden.
- **Costo**: cada acción cuesta 1.





Abstracción en la formulación (8-Puzzle)

- Toda formulación implica **abstracciones**.
- En el 8-Puzzle: las acciones se abstraen a **estado inicial/final**; se ignoran posiciones intermedias del deslizamiento.

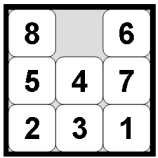
Ejemplo: 8-Puzzle (inicio y meta)

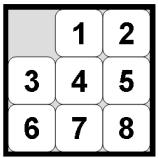
Inicio:

2	5	3
7	1	
4	8	6

Meta:

1	2	3
4	5	6
7	8	





Acciones: mover el **blanco** Izq/Der/Arr/Abj.
Transición: Izq en el inicio intercambia 5 y el blanco.

Section 1.3

Problemas del mundo real

Navegación de robots

- Generalización del problema de rutas.
- A diferencia de rutas fijas (carreteras), un robot **puede trazar sus propios caminos**.
- Robot circular en superficie plana: espacio de búsqueda 2D.
- Robots con brazos/piernas: espacio de búsqueda **multidimensional** (un eje por ángulo articular).

Retos en navegación robótica

- **Espacio continuo**: se requieren técnicas para volverlo finito (muestreo, discretización).
- **Errores de sensores/actuadores**: manejar ruido y deriva.
- **Observabilidad parcial**: información incompleta del entorno.
- **Entornos dinámicos**: otros agentes alteran el entorno.

Ejemplos de navegación

Robot circular (plano 2D):

- Acciones: avanzar, retroceder, izquierda, derecha, rotar.
- Meta: ir de un punto inicial a uno objetivo evitando obstáculos.

Robot con brazos/piernas:

- Acciones: controlar ángulos articulares.
- Meta: caminar, agarrar, trepar (espacio de estados de alta dimensión).

FIGURA
$\text{robot}_e xamples.png$



Table of Contents

2 Algoritmos de búsqueda

- Agentes que resuelven problemas
- Algoritmos de búsqueda

Algoritmos de búsqueda —idea general

- Un algoritmo de búsqueda recibe un problema y retorna **solución** o **fracaso**.
- Superpone un **árbol de búsqueda** sobre el grafo del espacio de estados.
- El árbol forma **caminos** desde el estado inicial hacia metas.
- Cada **nodo** del árbol corresponde a un **estado** del espacio.
- Cada **arco** del árbol corresponde a una **acción**.
- La **raíz** corresponde al **estado inicial**.

Espacio de estados vs. Árbol de búsqueda

Espacio de estados:

- Conjunto (posiblemente infinito) de estados y acciones del mundo.

Árbol de búsqueda:

- Describe **rutas** entre estados hacia la meta.
- Puede haber **múltiples caminos** al mismo estado.
- Cada nodo tiene una **ruta única** a la raíz.

FIGURA

$\text{state}_v s_t ree.png$

Ejemplo: Arad → Bucarest en el árbol

- **Raíz**: Arad.
- Expandir = considerar acciones y generar **hijos**.
- Hijos: Sibiu, Timișoara, Zerind (cada uno con **padre** Arad).

FIGURA
partial$_s earch_t rees.png$

Elegir qué nodo expandir

- Tras generar hijos, debemos elegir **cuál expandir** después.
- Esencia de la búsqueda: **seguir una opción ahora** y **postergar** otras.
- Ej.: expandir **Sibiu** primero ⇒ nueva **frontera** de nodos no expandidos.

FIGURA
frontier$_e$*xample.png*

Frontera del árbol de búsqueda

- La **frontera** es el conjunto de **nodos no expandidos**.
- Un estado está **alcanzado** si se generó un nodo para él.
- Un estado está **expandido** si se generaron sus hijos.

Grafo vs. Árbol (relación)

Cada nodo del árbol representa **todo un camino** en el grafo del espacio de estados.
Construimos **bajo demanda** y lo **mínimo** posible.

FIGURA
graph$_v s_t ree.png$

¿Qué tan grande es el árbol de búsqueda (desde S)?

Considera este grafo de 4 estados: analiza el **crecimiento** del árbol segun la **profundidad** y **ramificación**.

FIGURA
$four_state_graph.png$

Estructuras de datos para búsqueda

2 Algoritmos de búsqueda

Nodo del árbol:

- `STATE`: estado correspondiente.
- `PARENT`: nodo que lo generó.
- `ACTION`: acción aplicada al padre.
- `PATH-COST` $g(n)$: costo acumulado desde el inicio.

Recuperar el camino = seguir punteros `PARENT` hasta la raíz.

La frontera como cola (queue)

- Operaciones: IS-EMPTY, POP, TOP, ADD.
- Tipos:
  - **Cola de prioridad**: *best-first* (extrae el de menor $f(n)$).
  - **FIFO**: *breadth-first* (extrae el más antiguo).
  - **LIFO**/pila: *depth-first* (extrae el más reciente).

Estados alcanzados (tabla de *visited/reached*)

- Se almacenan en una **tabla** (p.ej., *hash*).
- Llave: **estado**; valor: **nodo** correspondiente.
- Ayuda a detectar **caminos redundantes** y conservar el **mejor** a cada estado.

Caminos redundantes y ciclos

2 Algoritmos de búsqueda

- **Camino redundante**: peor forma de llegar al mismo estado.
- Ejemplo hacia **Sibiu**:
  - Arad–Sibiu (140 millas).
  - Arad–Zerind–Oradea–Sibiu (297 millas).
- Los **ciclos** son caso especial de redundancia.
- Estrategias:
  - Recordar estados alcanzados y mantener el **mejor camino**.
  - Ignorar redundancias si son **raras/imposibles**.
  - Detectar **ciclos** (aunque no toda redundancia general).

Métricas de desempeño en resolución de problemas

2 Algoritmos de búsqueda

- **Completitud**: garantiza encontrar solución si existe.
- **Optimalidad de costo**: halla el **menor** costo de camino.
- **Complejidad temporal**: estados/acciones considerados.
- **Complejidad espacial**: memoria requerida.

FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Q&A

Thank you for your attention

Russell, S. J., & Norvig, P. (2016). Artificial intelligence: a modern approach. Pearson.
Hunt, E. B. (2014). Artificial intelligence. Academic Press.
Ertel, W. (2024). Introduction to artificial intelligence. Springer Nature.